

Integration Manual

for S32K1_S32M24X SPI Driver

Document Number: IM2SPIASRR21-11 Rev0000R3.0.0 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Building the driver	7
3.1 Build Options	7
3.1.1 GCC Compiler/Assembler/Linker Options	7
3.1.1.1 GCC Compiler Options	7
3.1.1.2 GCC Assembler Options	9
3.1.1.3 GCC Linker Options	10
3.1.2 IAR Compiler/Assembler/Linker Options	10
3.1.2.1 IAR Compiler Options	10
3.1.2.2 IAR Assembler Options	11
3.1.2.3 IAR Linker Options	12
3.1.3 GHS Compiler/Assembler/Linker Options	12
3.1.3.1 GHS Compiler Options	12
3.1.3.2 GHS Assembler Options	14
3.1.3.3 GHS Linker Options	14
3.2 Files required for compilation	15
3.3 Setting up the plugins	17
3.3.1 DMA configuration	18
4 Function calls to module	26
4.1 Function Calls during Start-up	26
4.2 Function Calls during Shutdown	26
4.3 Function Calls during Wake-up	26
5 Module requirements	27
5.1 Exclusive areas to be defined in BSW scheduler	27
5.2 Exclusive areas not available on this platform	33
5.3 Peripheral Hardware Requirements	34
5.4 ISR to configure within AutosarOS - dependencies	34
5.5 ISR Macro	34
5.5.1 Without an Operating System	34
5.5.1.1 Using Software Vector Mode	34
5.5.1.2 Using Hardware Vector Mode	35
5.5.2 With an Operating System	35
5.6 Other AUTOSAR modules - dependencies	35
5.7 Data Cache Restrictions	35
5.8 User Mode support	36
5.8.1 User Mode configuration in the module	36
5.8.2 User Mode configuration in AutosarOS	36
5.9 MultiPartition support	37
6 Main API Requirements	38
6.1 Main function calls within BSW scheduler	38
6.2 API Requirements	38
6.3 Calls to Notification Functions, Callbacks, Callouts	38
7 Memory allocation	39

7.1 Sections to be defined in Spi_MemMap.h	39
7.2 Linker command file	40
8 Integration Steps	41
9 External assumptions for driver	42

Chapter 1

Revision History

Revision	Date	Author	Description
1.0	07.03.2025	NXP RTD Team	S32K1_S32M24X Real-Time Drivers AUTOSAR R21-11 Version 3.0.0

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This integration manual describes the integration requirements for SPI Driver for S32K1_S32M24X microcontrollers.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32k116_qfn32
- s32k116_lqfp48
- s32k118_lqfp48
- s32k118_lqfp64
- s32k142_lqfp48
- s32k142_lqfp64
- s32k142_lqfp100
- s32k142w_lqfp48
- s32k142w_lqfp64
- s32k144_lqfp48
- s32k144_lqfp64 / MWCT1014S_lqfp64
- s32k144_lqfp100 / MWCT1014S_lqfp100
- s32k144_mapbga100
- s32k144w_lqfp48
- s32k144w_lqfp64
- s32k146_lqfp64
- s32k146_lqfp100 / MWCT1015S_lqfp100

- s32k146_mapbga100 / MWCT1015S_mapbga100
- s32k146_lqfp144
- s32k148_lqfp100
- s32k148_mapbga100 / MWCT1016S_mapbga100
- s32k148_lqfp144
- s32k148_lqfp176
- s32m241_lqfp64
- s32m242_lqfp64
- s32m243_lqfp64
- s32m244_lqfp64

All of the above microcontroller devices are collectively named as S32K1_S32M24X. Note: MWCT part numbers contain NXP confidential IP for Qi Wireless Power

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSMI	Basic Software Make file Interface
CS	Chip Select
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
FIFO	First In First Out
LSB	Least Significant Bit
MCU	Micro Controller Unit
MIDE	Multi Integrated Development Environment
MSB	Most Significant Bit
N/A	Not Applicable
RAM	Random Access Memory
SIU	Systems Integration Unit
SWS	Software Specification
SPI	Serial Peripheral Interface
XML	Extensible Markup Language
BSW	Basic Software
ISR	Interrupt Service Routine
OS	Operating System
GUI	Graphical User Interface
PB Variant	Post Build Variant
PC Variant	Pre Compile Variant
LT Variant	Link Time Variant

2.5 Reference List

#	Title	Version
1	Specification of Driver	AUTOSAR Release R21-11
2	S32K1xx Reference Manual	Rev.14.1, 01/2024
3	S32M24x Reference Manual	Rev. 5, 12/2024
4	S32K1xx Data Sheet	Rev. 14, 08/2021
5	S32M2xx Data Sheet	Rev. 7, 12/2024
6	S32K116 Errata	Mask Set Errata for Mask 0N96V Rev. 17/JAN/2024, 1/2024
7	S32K118 Errata	Mask Set Errata for Mask 0N97V Rev. 15/JAN/2024, 1/2024
8	S32K142 Errata	Mask Set Errata for Mask 0N33V Rev. 15/JAN/2024, 1/2024
9	S32K144 Errata	Mask Set Errata for Mask 0N57U Rev. 15/JAN/2024, 1/2024
10	S32K144W Errata	Mask Set Errata for Mask 0P64A Rev. 03/JUL/2024, 7/2024
11	S32K146 Errata	Mask Set Errata for Mask 0N73V Rev. 15/JAN/2024, 1/2024

Introduction

#	Title	Version
12	S32K148 Errata	Mask Set Errata for Mask 0N20V Rev. 15/JAN/2024, 1/2024
13	S32M242 Errata	Mask Set Errata for Mask N33V+P69K Rev.1, 8/2024
14	S32M244 Errata	Mask Set Errata for Mask P64A+P69K Rev.1, 8/2024

Chapter 3

Building the driver

- [Build Options](#)
- [Files required for compilation](#)
- [Setting up the plugins](#)

This section describes the source files and various compilers, linker options used for building the driver. It also explains the EB Tresos Studio plugin setup procedure.

3.1 Build Options

- [GCC Compiler/Assembler/Linker Options](#)
- [IAR Compiler/Assembler/Linker Options](#)
- [GHS Compiler/Assembler/Linker Options](#)

The RTD driver files are compiled using:

- NXP GCC 10.2.0 20200723 (Build 1728 Revision g5963bc8)
- IAR ANSI C/C++ Compiler V8.40.3.228/W32 for ARM Functional Safety
- Green Hills Multi 7.1.6d / Compiler 2020.1.4

The compiler, assembler, and linker flags used for building the driver are explained below.

The TS_T40D2M30I0R0 part of the plugin name is composed as follows:

- T = Target_Id (e.g. T40 identifies Cortex-M architecture)
- D = Derivative_Id (e.g. D2 identifies S32K1 platform)
- M = SW_Version_Major and SW_Version_Minor
- I = SW_Version_Patch
- R = Reserved

3.1.1 GCC Compiler/Assembler/Linker Options

3.1.1.1 GCC Compiler Options

Compiler Option	Description
-mcpu=cortex-m4	Targeted ARM processor for which GCC should tune the performance of the code (for S32K14x or S32M24x devices)

Compiler Option	Description
-mcpu=cortex-m0plus	Targeted ARM processor for which GCC should tune the performance of the code (for S32K11x devices)
-mthumb	Generates code that executes in Thumb state
-mlittle-endian	Generate code for a processor running in little-endian mode
-mfpv4-sp-d16	Specifies the floating-point hardware available on the target (for S32K14x or S32M24x devices)
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions (for S32K14x or S32M24x devices)
-mfpv4-sp-d16	Specifies the floating-point hardware available on the target (for S32K11x devices)
-mfloat-abi=soft	Specifies the floating-point ABI to use. Specifying "soft" causes GCC to generate output containing library calls for floating-point operations (for S32K11x devices)
-std=c99	Specifies the ISO C99 base standard
-Os	Optimize for size. Enables all -O2 optimizations except those that often increase code size
-ggdb3	Produce debugging information for use by GDB using the most expressive format available, including GDB extensions if at all possible. Level 3 includes extra information, such as all the macro definitions present in the program
-Wall	Enables all the warnings about constructions that some users consider questionable, and that are easy to avoid (or modify to prevent the warning), even in conjunction with macros
-Wextra	This enables some extra warning flags that are not enabled by -Wall
-pedantic	Issue all the warnings demanded by strict ISO C. Reject all programs that use forbidden extensions. Follows the version of the ISO C standard specified by the aforementioned -std option
-Wstrict-prototypes	Warn if a function is declared or defined without specifying the argument types
-Wundef	Warn if an undefined identifier is evaluated in an #if directive. Such identifiers are replaced with zero
-Wunused	Warn whenever a function, variable, label, value, macro is unused
-Werror=implicit-function-declaration	Make the specified warning into an error. This option throws an error when a function is used before being declared
-Wsign-compare	Warn when a comparison between signed and unsigned values could produce an incorrect result when the signed value is converted to unsigned.
-Wdouble-promotion	Give a warning when a value of type float is implicitly promoted to double
-fno-short-enums	Specifies that the size of an enumeration type is at least 32 bits regardless of the size of the enumerator values.
-funsigned-char	Let the type char be unsigned by default, when the declaration does not use either signed or unsigned
-funsigned-bitfields	Let a bit-field be unsigned by default, when the declaration does not use either signed or unsigned

Compiler Option	Description
-fomit-frame-pointer	Omit the frame pointer in functions that don't need one. This avoids the instructions to save, set up and restore the frame pointer; on many targets it also makes an extra register available.
-fno-common	Makes the compiler place uninitialized global variables in the BSS section of the object file. This inhibits the merging of tentative definitions by the linker so you get a multiple-definition error if the same variable is accidentally defined in more than one compilation unit
-fstack-usage	This option is only used to build test for generation Ram/↔ Stack size report. Makes the compiler output stack usage information for the program, on a per-function basis
-fdump-ipa-all	This option is only used to build test for generation Ram/↔ Stack size report. Enables all inter-procedural analysis dumps
-c	Stop after assembly and produce an object file for each source file
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initialization in source file system.c under the Platform driver (for S32K14x or S32↔M24x devices)
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initialization in source file system.c under the Platform driver (for S32K14x or S32M24x devices)
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPORT↔ as a macro, with definition 1. Allows drivers to be configured in user mode.
-sysroot=	Specifies the path to the sysroot, for Cortex-M7 it is /arm-none-eabi/newlib
-specs=nano.specs	Use Newlib nano specs
-specs=nosys.specs	Do not use printf/scanf

3.1.1.2 GCC Assembler Options

Assembler Option	Description
-xassembler-with-cpp	Specifies the language for the following input files (rather than letting the compiler choose a default based on the file name suffix)
-mcpu=cortex-m4	Targeted ARM processor for which GCC should tune the performance of the code (for S32K14x or S32M24x devices)
-mcpu=cortex-m0plus	Targeted ARM processor for which GCC should tune the performance of the code (for S32K11x devices)
-mfpu=fpv4-sp-d16	Specifies the floating-point hardware available on the target (for S32K14x devices)
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions (for S32K14x devices)
-mfpu=auto	Specifies the floating-point hardware available on the target (for S32K11x devices)

Building the driver

Assembler Option	Description
-mfloat-abi=soft	Specifies the floating-point ABI to use. Specifying "soft" causes GCC to generate output containing library calls for floating-point operations (for S32K11x devices)
-mthumb	Generates code that executes in Thumb state
-c	Stop after assembly and produce an object file for each source file

3.1.1.3 GCC Linker Options

Linker Option	Description
-Wl,-Map,filename	Produces a map file
-T linkerfile	Use linkerfile as the linker script. This script replaces the default linker script (rather than adding to it)
-entry=Reset_Handler	Specifies that the program entry point is Reset_Handler
-nostartfiles	Do not use the standard system startup files when linking
-mcpu=cortex-m4	Targeted ARM processor for which GCC should tune the performance of the code (for S32K14x or S32M24x devices)
-mcpu=cortex-m0plus	Targeted ARM processor for which GCC should tune the performance of the code (for S32K11x devices)
-mthumb	Generates code that executes in Thumb state
-mfpu=fpv4-sp-d16	Specifies the floating-point hardware available on the target (for S32K14x or S32M24x devices)
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions (for S32K14x or S32M24x devices)
-mfpu=auto	Specifies the floating-point hardware available on the target (for S32K11x devices)
-mfloat-abi=soft	Specifies the floating-point ABI to use. Specifying "soft" causes GCC to generate output containing library calls for floating-point operations (for S32K11x devices)
-mlittle-endian	Generate code for a processor running in little-endian mode
-ggdb3	Produce debugging information for use by GDB using the most expressive format available, including GDB extensions if at all possible. Level 3 includes extra information, such as all the macro definitions present in the program
-lc	Link with the C library
-lm	Link with the Math library
-lgcc	Link with the GCC library
-n	Turn off page alignment of sections, and disable linking against shared libraries
-sysroot=	Specifies the path to the sysroot, for Cortex-M7 it is /arm-none-eabi/newlib
-specs=nano.specs	Use Newlib nano specs
-specs=nosys.specs	Do not use printf/scanf

3.1.2 IAR Compiler/Assembler/Linker Options

3.1.2.1 IAR Compiler Options

Compiler Option	Description
-cpu=Cortex-M4	Targeted ARM processor for which IAR should tune the performance of the code (for S32K14x or S32M24x devices)

Compiler Option	Description
-cpu=Cortex-M0+	Targeted ARM processor for which IAR should tune the performance of the code (for S32K11x devices)
-cpu_mode=thumb	Generates code that executes in Thumb state
-endian=little	Generate code for a processor running in little-endian mode
-fpu=FPv4-SP	Use this option to generate code that performs floating-point operations using a Floating Point Unit (FPU). Single-precision variant. (for S32K14x or S32M24x devices)
-fpu=none	Use this option to generate code that performs floating-point operations using a Floating Point Unit (FPU). No FPU. (for S32K11x devices)
-e	Enables all IAR C language extensions
-Ohz	Optimize for size. The compiler will emit AEABI attributes indicating the requested optimization goal. This information can be used by the linker to select smaller or faster variants of DLIB library functions
-debug	Makes the compiler include debugging information in the object modules. Including debug information will make the object files larger
-no_clustering	Disables static clustering optimizations. Static and global variables defined within the same module will not be arranged so that variables that are accessed in the same function are close to each other
-no_mem_idioms	Makes the compiler not optimize certain memory access patterns
-no_explicit_zero_opt	Do not treat explicit initializations to zero of static variables as zero initializations
-require_prototypes	Force the compiler to verify that all functions have proper prototypes. Generates an error otherwise
-no_wrap_diagnostics	Does not wrap long lines in diagnostic messages
-diag_suppress=Pa050	Suppresses diagnostic message Pa050
-DIAR	Predefine IAR as a macro, with definition 1
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode.
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initialization in source file system.c under the Platform driver (for S32K14x or S32M24x devices)
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initialization in source file system.c under the Platform driver (for S32K14x or S32M24x devices)
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPORT as a macro, with definition 1. Allows drivers to be configured in user mode.

3.1.2.2 IAR Assembler Options

Assembler Option	Description
-cpu=Cortex-M4	Targeted ARM processor for which IAR should tune the performance of the code (for S32K14x or S32M24x devices)
-cpu=Cortex-M0+	Targeted ARM processor for which IAR should tune the performance of the code (for S32K11x devices)
-fpu=FPv4-SP	Use this option to generate code that performs floating-point operations using a Floating Point Unit (FPU). Single-precision variant. (for S32K14x devices)
-fpu=none	Use this option to generate code that performs floating-point operations using a Floating Point Unit (FPU). No FPU. (for S32K11x devices)
-cpu_mode thumb	Selects the thumb mode for the assembler directive CODE
-g	Disables the automatic search for system include files
-r	Generates debug information

3.1.2.3 IAR Linker Options

Linker Option	Description
-map filename	Produces a map file
-config linkerfile	Use linkerfile as the linker script. This script replaces the default linker script (rather than adding to it)
-cpu=Cortex-M4	Targeted ARM processor for which IAR should tune the performance of the code (for S32K14x or S32M24x devices)
-cpu=Cortex-M0+	Targeted ARM processor for which IAR should tune the performance of the code (for S32K11x devices)
-fpu=FPv4-SP	Use this option to generate code that performs floating-point operations using a Floating Point Unit (FPU). Single-precision variant. (for S32K14x or S32M24x devices)
-fpu=none	Use this option to generate code that performs floating-point operations using a Floating Point Unit (FPU). No FPU. (for S32K11x devices)
-entry __start	Treats __start as a root symbol and start label
-enable_stack_usage	Enables stack usage analysis. If a linker map file is produced, a stack usage chapter is included in the map file
-skip_dynamic_initialization	Dynamic initialization (typically initialization of C++ objects with static storage duration) will not be performed automatically during application startup
-no_wrap_diagnostics	Does not wrap long lines in diagnostic messages

3.1.3 GHS Compiler/Assembler/Linker Options

3.1.3.1 GHS Compiler Options

Compiler Option	Description
-cpu=cortexm4	Selects target processor: Arm Cortex M4 (for S32K14x or S32M24x devices)
-cpu=cortexm0plus	Selects target processor: Arm Cortex M0+ (for S32K11x devices)
-thumb	Selects generating code that executes in Thumb state

Compiler Option	Description
-fpu=vfpv4_d16	Specifies hardware floating-point using the v4 version of the VFP instruction set, with 16 double-precision floating-point registers (for S32K14x or S32M24x devices)
-fsingle	Use hardware single-precision, software double-precision FP instructions (for S32K14x or S32M24x devices)
-fsoft	Specifies software floating-point (SFP) mode. This setting causes your target to use integer registers to hold floating-point data and use library subroutine calls to emulate floating-point operations (for S32K11x devices)
-C99	Use (strict ISO) C99 standard (without extensions)
-ghstd=last	Use the most recent version of Green Hills Standard mode (which enables warnings and errors that enforce a stricter coding standard than regular C and C++)
-Osize	Optimize for size
-gnu_asm	Enables GNU extended asm syntax support
-dual_debug	Generate DWARF 2.0 debug information
-G	Generate debug information
-keeptempfiles	Prevents the deletion of temporary files after they are used. If an assembly language file is created by the compiler, this option will place it in the current directory instead of the temporary directory
-Wimplicit-int	Produce warnings if functions are assumed to return int
-Wshadow	Produce warnings if variables are shadowed
-Wtrigraphs	Produce warnings if trigraphs are detected
-Wundef	Produce a warning if undefined identifiers are used in #if preprocessor statements
-unsigned_chars	Let the type char be unsigned, like unsigned char
-unsigned_fields	Bitfields declared with an integer type are unsigned
-no_commons	Allocates uninitialized global variables to a section and initializes them to zero at program startup
-no_exceptions	Disables C++ support for exception handling
-no_slash_comment	C++ style // comments are not accepted and generate errors
-prototype_errors	Controls the treatment of functions referenced or called when no prototype has been provided
-incorrect_pragma_warnings	Controls the treatment of valid #pragma directives that use the wrong syntax
-c	Stop after assembly and produce an object file for each source file
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initialization in source file system.c under the Platform driver (for S32K14x or S32M24x devices)

Compiler Option	Description
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initialization in source file system.c under the Platform driver (for S32K14x or S32M24x devices)
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPORT as a macro, with definition 1. Allows drivers to be configured in user mode

3.1.3.2 GHS Assembler Options

Assembler Option	Description
-cpu=cortexm4	Selects target processor: Arm Cortex M4 (for S32K14x or S32M24x devices)
-cpu=cortexm0plus	Selects target processor: Arm Cortex M0+ (for S32K11x devices)
-fpu=vfpv4_d16	Specifies hardware floating-point using the v4 version of the VFP instruction set, with 16 double-precision floating-point registers (for S32K14x devices)
-fsingle	Use hardware single-precision, software double-precision FP instructions (for S32K14x devices)
-fsoft	Specifies software floating-point (SFP) mode. This setting causes your target to use integer registers to hold floating-point data and use library subroutine calls to emulate floating-point operations (for S32K11x devices)
-preprocess_assembly_files	Controls whether assembly files with standard extensions such as .s and .asm are preprocessed
-list	Creates a listing by using the name and directory of the object file with the .lst extension
-c	Stop after assembly and produce an object file for each source file

3.1.3.3 GHS Linker Options

Linker Option	Description
-e Reset_Handler	Make the symbol Reset_Handler be treated as a root symbol and the start label of the application
-T linker_script_file.ld	Use linker_script_file.ld as the linker script. This script replaces the default linker script (rather than adding to it)
-map	Produce a map file
-keepmap	Controls the retention of the map file in the event of a link error
-Mn	Generates a listing of symbols sorted alphabetically/numerically by address
-delete	Instructs the linker to remove functions that are not referenced in the final executable. The linker iterates to find functions that do not have relocations pointing to them and eliminates them
-ignore_debug_references	Ignores relocations from DWARF debug sections when using -delete. DWARF debug information will contain references to deleted functions that may break some third-party debuggers
-Llibrary_path	Points to library_path (the libraries location) for thumb2 to be used for linking
-larch	Link architecture specific library
-lstartup	Link run-time environment startup routines. The source code for the modules in this library is provided in the src/libstartup directory

Linker Option	Description
-lind_sd	Link language-independent library, containing support routines for features such as software floating point, run-time error checking, C99 complex numbers, and some general purpose routines of the ANSI C library (for S32K14x or S32M24x devices)
-lind_sf	Link language-independent library, containing support routines for features such as software floating point, run-time error checking, C99 complex numbers, and some general purpose routines of the ANSI C library (for S32K11x devices)
-v	Prints verbose information about the activities of the linker, including the libraries it searches to resolve undefined symbols
-keep=C40_Ip_AccessCode	Avoid linker remove function C40_Ip_AccessCode from Fls module because it is not referenced explicitly
-nostartfiles	Controls the start files to be linked into the executable

3.2 Files required for compilation

This section describes the include files required to compile, assemble and link the AUTOSAR Spi Driver for S32K1XX microcontrollers.

To avoid integration of incompatible files, all the include files from other modules shall have the same AR_MAJOR_VERSION and AR_MINOR_VERSION, i.e. only files with the same AUTOSAR major and minor versions can be compiled.

Spi Driver Files:

- Spi_TS_T40D2M30I0R0\src\Lpspi_Ip.c
- Spi_TS_T40D2M30I0R0\src\Lpspi_Ip_Irq.c
- Spi_TS_T40D2M30I0R0\src\Flexio_Spi_Ip.c
- Spi_TS_T40D2M30I0R0\src\Flexio_Spi_Ip_Irq.c
- Spi_TS_T40D2M30I0R0\src\Spi.c
- Spi_TS_T40D2M30I0R0\src\Spi_IPW.c
- Spi_TS_T40D2M30I0R0\inc\Lpspi_Ip.h
- Spi_TS_T40D2M30I0R0\inc\Lpspi_Ip_Types.h
- Spi_TS_T40D2M30I0R0\inc\Flexio_Spi_Ip.h
- Spi_TS_T40D2M30I0R0\inc\Flexio_Spi_Ip_Irq.h
- Spi_TS_T40D2M30I0R0\inc\Flexio_Spi_Ip_Types.h
- Spi_TS_T40D2M30I0R0\inc\Spi.h
- Spi_TS_T40D2M30I0R0\inc\Spi_IPW.h
- Spi_TS_T40D2M30I0R0\inc\Spi_IPW_Types.h

Spi Driver Generated Files (must be generated by the user using a configuration tool):

- Spi_[VariantName]_PBcfg.c (For PB Variant) - The file contains the definition of the initialization pointer for the respective variant.
- Spi_Ipw_[VariantName]_PBcfg.c - The file contains the definition of the initialization pointer for the respective variant.
- Lpspi_Ip_[VariantName]_PBcfg.c - The file contains the definition of the initialization pointer for the respective variant.

Building the driver

- Flexio_Spi_Ip_[VariantName]_PBcfg.c - The file contains the definition of the initialization pointer for the respective variant.
- Spi_[VariantName]_PBcfg.h - The file contains externing definitions of configuration structures.
- Spi_Ipw_[VariantName]_PBcfg.h - The file externing definitions of configuration structures.
- Spi_Ipw_Cfg.h - The file contains the definitions according to configuration.
- Spi_Cfg.h - The file contains the definitions according to configuration.
- Lpspi_Ip_[VariantName]_PBcfg.h - The file externing definitions of configuration structures.
- Flexio_Spi_Ip_[VariantName]_PBcfg.h - The file externing definitions of configuration structures.
- Lpspi_Ip_Cfg.h - The file contains the definitions according to configuration.
- Flexio_Spi_Ip_Cfg.h - The file contains the definitions according to configuration.

Note

As a deviation from standard:

- Spi_[VariantName]_PBcfg.c, Spi_Ipw_[VariantName]_PBcfg.c, Lpspi_Ip_[VariantName]_PBcfg.c, Flexio_Spi_Ip_[VariantName]_PBcfg.c, Spi_[VariantName]_PBcfg.h, Spi_Ipw_[VariantName]_PBcfg.h, Lpspi_Ip_[VariantName]_PBcfg.h, Flexio_Spi_Ip_[VariantName]_PBcfg.h - These files will contain the definition for all parameters that are variant aware, independent of the configuration class that will be selected (PC, LT, PB).

Base Files:

- Base_TS_T40D2M30I0R0\include\Compiler.h
- Base_TS_T40D2M30I0R0\include\Compiler_Cfg.h
- Base_TS_T40D2M30I0R0\include\ComStack_Cfg.h
- Base_TS_T40D2M30I0R0\include\ComStack_Types.h
- Base_TS_T40D2M30I0R0\include\Mcal.h
- Base_TS_T40D2M30I0R0\include\Spi_MemMap.h
- Base_TS_T40D2M30I0R0\include\Platform_Types.h
- Base_TS_T40D2M30I0R0\include\Reg_eSys.h
- Base_TS_T40D2M30I0R0\include\RegLockMacros.h
- Base_TS_T40D2M30I0R0\include\Soc_Ips.h
- Base_TS_T40D2M30I0R0\include\StandardTypes.h
- Base_TS_T40D2M30I0R0\include\OsIf.h
- Base_TS_T40D2M30I0R0\include\Devassert.h
- Base_TS_T40D2M30I0R0\header\[PlatformName]_LPSPI.h
- Base_TS_T40D2M30I0R0\header\[PlatformName]_FLEXIO.h

DEM Files:

- Dem_TS_T40D2M30I0R0\include\Dem.h
- Dem_TS_T40D2M30I0R0\include\Dem_Types.h
- Dem_TS_T40D2M30I0R0\generate_PC\include\Dem_IntErrId.h
- Dem_TS_T40D2M30I0R0\src\Dem.c

DET Files:

- Det_TS_T40D2M30I0R0\include\Det.h
- Det_TS_T40D2M30I0R0\src\Det.c

MCL Files(when at least DMA or SPI over FLEXIO is used):

- Mcl_TS_T40D2M30I0R0\include\CDD_Mcl.h
- Mcl_TS_T40D2M30I0R0\src\CDD_Mcl.c

RTE Files:

- Rte_TS_T40D2M30I0R0\include\SchM_Spi.h
- Rte_TS_T40D2M30I0R0\src\SchM_Spi.c

3.3 Setting up the plugins

The Spi Driver was designed to be configured by using the EB Tresos Studio (version 29.0.0 b220329-0119 or later)

Location of various files inside the SPI module folder:

- VSMD (Vendor Specific Module Definition) file in EB Tresos Studio XDM format:
 - Spi_TS_T40D2M30I0R0\config\Spi.xdm
- VSMD (Vendor Specific Module Definition) file(s) in AUTOSAR compliant EPD format:
 - Spi_TS_T40D2M30I0R0\autosar\Spi_<subderivative_name>.epd
- Code Generation Templates for variant aware parameters:
 - Spi_TS_T40D2M30I0R0\generate_PB\src\Spi_PBcfg.c
 - Spi_TS_T40D2M30I0R0\generate_PB\src\Spi_Ipw_PBcfg.c
 - Spi_TS_T40D2M30I0R0\generate_PB\src\Spi_PBcfg.c
 - Spi_TS_T40D2M30I0R0\generate_PB\src\Spi_PBcfg.h
 - Spi_TS_T40D2M30I0R0\generate_PB\src\Spi_Ipw_PBcfg.h
 - Spi_TS_T40D2M30I0R0\generate_PB\src\Lpspi_Ip_PBcfg.h
 - Spi_TS_T40D2M30I0R0\generate_PB\src\Flexio_Spi_Ip_PBcfg.h
- Code Generation Templates for parameters without variation points:
 - Spi_TS_T40D2M30I0R0\generate_PC\include\Spi_Cfg.h
 - Spi_TS_T40D2M30I0R0\generate_PC\include\Spi_Ipw_Cfg.h
 - Spi_TS_T40D2M30I0R0\generate_PC\include\Lpspi_Ip_Cfg.h
 - Spi_TS_T40D2M30I0R0\generate_PC\include\Flexio_Spi_Ip_Cfg.h

Steps to generate the configuration:

1. Copy the following module folders into the Tresos plugins folder:
 - BaseNXP_TS_T40D2M30I0R0
 - Dem_TS_T40D2M30I0R0
 - Det_TS_T40D2M30I0R0

- EcuC_TS_T40D2M30I0R0
 - Mcl_TS_T40D2M30I0R0
 - Mcu_TS_T40D2M30I0R0
 - Os_TS_T40D2M30I0R0
 - Platform_TS_T40D2M30I0R0
 - Port_TS_T40D2M30I0R0
 - Resource_TS_T40D2M30I0R0
 - Rm_TS_T40D2M30I0R0
 - Rte_TS_T40D2M30I0R0
 - Spi_TS_T40D2M30I0R0
2. Set the desired Tresos Output location folder for the generated sources and header files.
 3. Use the EB Tresos Studio GUI to modify ECU configuration parameters values.
 4. Generate the configuration files

Dependencies:

- **DEM** is required for signalling the production error detection (hardware failure, etc).
- **DET** is required for signalling the development error detection (parameters out of range, null pointers, etc).
- **ECUC** is required for configuring the variant handling and partition in Tresos.
- **MCL** is required when DMA option is used or HW Unit is SPI over FLEXIO.
- **MCU** is required to generate baudrate according to Peripheral clock.
- **OS** is required for configuring the core selected in EB Tresos and the GetUserId API has called in SPI driver.
- **PLATFORM** is required for configuring the Spi irq handler.
- **PORT** is required for configuring the Spi pins.
- **RESOURCE** is required to select processor derivative.
- **RM** is required for configuring the DMA MUX.

DMA configuration

3.3.1 DMA configuration

This section applies only to SPI units configured for asynchronous transmission (SpiPhyUnitSync not checked) and which use DMA for serializing/deserializing data between the hardware unit and the TX/RX buffers (SpiPhyUnit← AsyncUseDma = true).

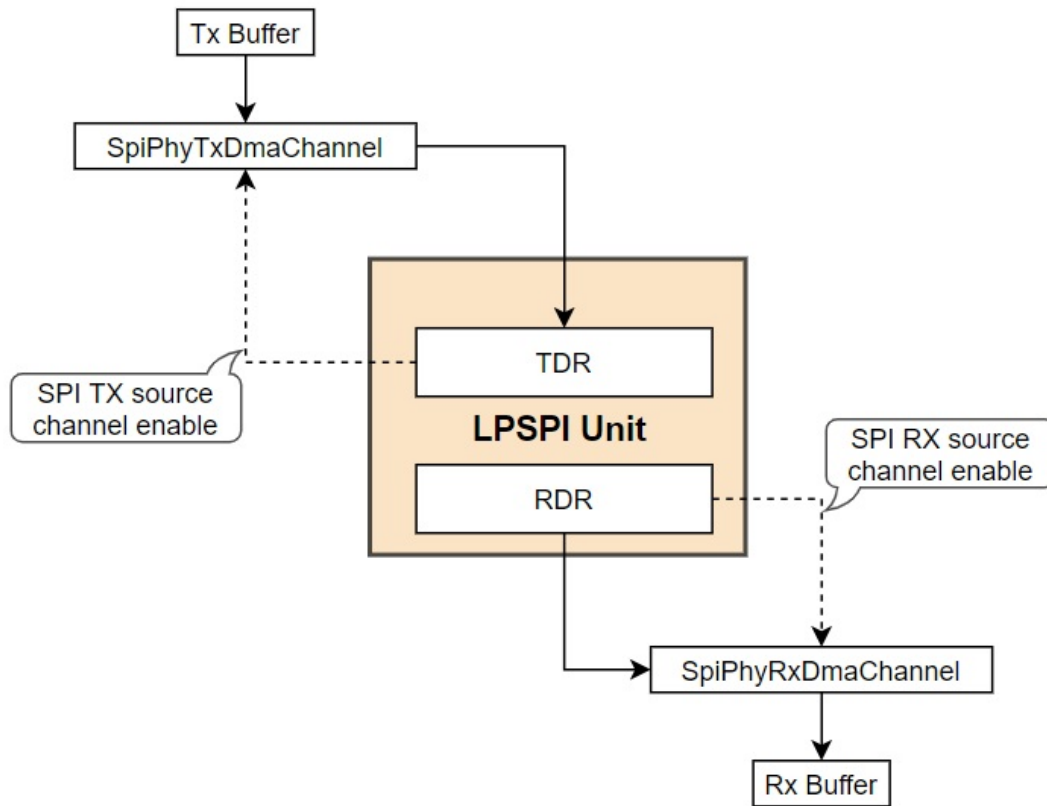


Figure 3.1 DMA transferring mode internal architecture

The mechanism in this diagram is also applied for FLEXIO but SHIFTBUFs used instead of TDR, RDR. Each SPI unit configured in DMA mode requires 2 distinct DMA channels from the same DMA Mux:

- **SpiPhyTxDmaChannel:** The TX DMA channel used for filling TX FIFO with the data in TX Buffer. This channel is triggered by TX SPI unit event and must be “wired” to “SPI TX source” (configured inside the MCL module – "McConfig/Dma Logic Channel" container).
- **SpiPhyRxDmaChannel:** The RX DMA channel used for filling RX buffer with the deserialized data. This channel is triggered by RX SPI unit event and must be “wired” to “SPI RX source” (configured inside the MCL module – "McConfig/Dma Logic Channel" container).

Physical Unit	SPI TX DMA Notification Name	SPI RX DMA Notification Name
---------------	------------------------------	------------------------------

Note

- If DMA uses fixed priority arbitration, then the priority must be SpiPhyRxDmaChannel > SpiPhyTx↔DmaChannel.
- If DMA uses round robin arbitration, no priority constraints are applied on SpiPhyTxDmaChannel and SpiPhyRxDmaChannel priority.

If the SPI driver is working in interrupt mode, the DMA Tx and DMA Rx notification must be enabled for the specified Tx DMA and Rx DMA channels. The name of the function to be used as a notification is Lpspi_Ip_L↔PSPI_X_IrqTxDmaHandler, Flexio_Spi_Ip_FLEXIO_SPI_X_IrqTxDmaHandler or Lpspi_Ip_LPSPI_X_Irq↔RxDmaHandler, Flexio_Spi_Ip_FLEXIO_SPI_X_IrqRxDmaHandler where X is the number of SPI unit used.

Table 3.10 SPI DMA Notification

Physical Unit	SPI TX DMA Notification Name	SPI RX DMA Notification Name
LPSPi_0	Lpspi_Ip_LPSPi_0_IrqTxDmaHandler	Lpspi_Ip_LPSPi_0_IrqRxDmaHandler
LPSPi_1	Lpspi_Ip_LPSPi_1_IrqTxDmaHandler	Lpspi_Ip_LPSPi_1_IrqRxDmaHandler
LPSPi_2	Lpspi_Ip_LPSPi_2_IrqTxDmaHandler	Lpspi_Ip_LPSPi_2_IrqRxDmaHandler
FLEXIO_SPI↔_0	Flexio_Spi_Ip_FLEXIO_SPI_0_IrqTx↔DmaHandler	Flexio_Spi_Ip_FLEXIO_SPI_0_IrqRx↔DmaHandler
FLEXIO_SPI↔_1	Flexio_Spi_Ip_FLEXIO_SPI_1_IrqTx↔DmaHandler	Flexio_Spi_Ip_FLEXIO_SPI_1_IrqRx↔DmaHandler

Next figures show an example of DMA configuration for SPI unit.

SpiPhyUnit

Name  SpiPhyUnit_0

General

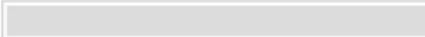
SpiPhyUnitMapping	 LPSPi_2
SpiPhyUnitMode	 SPI_MASTER
SpiPhyUnitSync	 ☐ 
SpiSamplePoint (0 -> 1)	 0
SpiPinConfiguration (0 -> 3)	 0
SpiPhyUnitClockRef	 /Mcu/Mcu/McuModuleConfiguration/Mcu
 SpiPhyUnitAlternateClockRef	 
SpiPhyUnitAsyncUseDma	 ☒ 
 SpiPhyTxDmaChannel	 /Mcl/Mcl/MclConfig/DMA_LPSPi2_TX
 SpiPhyRxDmaChannel	 /Mcl/Mcl/MclConfig/DMA_LPSPi2_RX

Figure 3.2 DMA Configuration sample for SPI Physical Unit - SPI module in EB Tresos

Logic Channel

Name  DMA_LPSPi2_TX

Logic Channel Configuration Global Transfer ScatterGather

Logic Channel Name  DMA_LOGIC_CH_0 

Hardware Instance  DMA_IP_HW_INST_0  

Hardware Channel  DMA_IP_HW_CH_0  

Interrupt Callback  Lpspi_lp_LPSPi2_IrqTxDmaHandler 

Error Interrupt Callback  NULL_PTR 

Ecuc Partition Ref  

Enable Global Config  ☒  Enable Transfer Config  ☐ 

Enable Scatter/Gather  ☐ 

Figure 3.3 DMA TX General configuration - MCL module in EB Tresos

Logic Channel

Name  DMA_LPSPi2_TX

Logic Channel Configuration Global Transfer ScatterGather

Name  dmaLogicChannel_GlobalConfigType

▼ Control

Name  dmaLogicChannelConfig_GlobalControlType

Enable Master Id Replication  ☐  Enable Buffered Writes  ☐ 

▼ Request

Name  dmaLogicChannelConfig_GlobalRequestType

Enable DMAMUX Trigger  ☐  Enable DMAMUX Source  ☒ 

DMAMUX0 Source  DMA_IP_REQ_MUX0_LPSPi2_TX 

DMAMUX1 Source  DMA_IP_REQ_MUX1_DISABLED 

Enable DMA Request  ☐ 

Figure 3.4 DMA TX Global Configuration - MCL module in EB Tresos

Logic Channel

Name

DMA_LPSPi2_RX

Logic Channel Configuration

Global

Transfer

ScatterGather

Logic Channel Name

DMA_LOGIC_CH_1

Hardware Instance

DMA_IP_HW_INST_0

Hardware Channel

DMA_IP_HW_CH_1

Interrupt Callback

Lpspi_lp_LPSPi2_IrqRxDmaHandler

Error Interrupt Callback

NULL_PTR

Ecuc Partition Ref

Enable Global Config

☒

Enable Transfer Config

☐

Enable Scatter/Gather

☐

Figure 3.5 DMA RX General configuration - MCL module in EB Tresos

Logic Channel

Name  DMA_LPSP12_RX

Logic Channel Configuration Global Transfer ScatterGather

▼ dmaLogicChannel_GlobalConfigType

Name  dmaLogicChannel_GlobalConfigType

▼ Control

Name  dmaLogicChannelConfig_GlobalControlType

Enable Master Id Replication  ☐  Enable Buffered Writes  ☐ 

▼ Request

Name  dmaLogicChannelConfig_GlobalRequestType

Enable DMAMUX Trigger  ☐  Enable DMAMUX Source  ☒ 

DMAMUX0 Source  DMA_IP_REQ_MUX0_LPSP12_RX 

DMAMUX1 Source  DMA_IP_REQ_MUX1_DISABLED 

Enable DMA Request  ☐ 

Figure 3.6 DMA RX Global Configuration - MCL module in EB Tresos

Chapter 4

Function calls to module

- [Function Calls during Start-up](#)
- [Function Calls during Shutdown](#)
- [Function Calls during Wake-up](#)

4.1 Function Calls during Start-up

SPI shall be initialized during STARTUP phase of EcuM initialization. The API to be called for this is `Spi_↔Init()`. The MCU module should be initialized before the SPI is initialized. The API to be called for this purpose is `Spi_Init()`. The PORT and MCL (if the DMA option is used) modules shall be initialized before SPI is initialized.

4.2 Function Calls during Shutdown

SPI can be silenced by calling `Spi_DeInit()`.

4.3 Function Calls during Wake-up

N/A

Chapter 5

Module requirements

- Exclusive areas to be defined in BSW scheduler
- Exclusive areas not available on this platform
- Peripheral Hardware Requirements
- ISR to configure within AutosarOS - dependencies
- ISR Macro
- Other AUTOSAR modules - dependencies
- Data Cache Restrictions
- User Mode support
- MultiPartition support

5.1 Exclusive areas to be defined in BSW scheduler

SPI_EXCLUSIVE_AREA_00 is used in function Spi_AsyncTransmit to protect the schedule mechanism for the situation when a scheduling operation determined by a pending Spi_AsyncTransmit() call may be preempted by a job scheduling requested by an ISR event. It also protect concurrent Spi_AsyncTransmit() calls to schedule in the same time different jobs on the same SPI unit.

SPI_EXCLUSIVE_AREA_01 is used in function Spi_AsyncTransmit to protect the schedule mechanism for the situation when a scheduling operation determined by a pending Spi_AsyncTransmit() call may be preempted by a job scheduling requested by an ISR event.

SPI_EXCLUSIVE_AREA_02 is used in function Spi_Cancel to protect the status of the given Job result during Spi_Cancel called and ISR event.

SPI_EXCLUSIVE_AREA_02 is used in function Lpspi_Ip_LPSPI_0_IRQHandler to protect the status of the given Job result during Spi_Cancel called and ISR event.

SPI_EXCLUSIVE_AREA_02 is used in function Lpspi_Ip_LPSPI_1_IRQHandler to protect the status of the given Job result during Spi_Cancel called and ISR event.

SPI_EXCLUSIVE_AREA_02 is used in function Lpspi_Ip_LPSPI_2_IRQHandler to protect the status of the given Job result during Spi_Cancel called and ISR event.

SPI_EXCLUSIVE_AREA_02 is used in function Lpspi_Ip_LPSPI_3_IRQHandler to protect the status of the given Job result during Spi_Cancel called and ISR event.

SPI_EXCLUSIVE_AREA_02 is used in function Lpspi_Ip_LPSPI_4_IRQHandler to protect the status of the given Job result during Spi_Cancel called and ISR event.

SPI_EXCLUSIVE_AREA_02 is used in function `Lpspi_Ip_LPSPI_5_IRQHandler` to protect the status of the given Job result during `Spi_Cancel` called and ISR event.

SPI_EXCLUSIVE_AREA_03 is used in function `Spi_AsyncTransmit` to guarantee the atomicity of locking for the entire set of jobs belonging to an asynchronous sequence.

SPI_EXCLUSIVE_AREA_04 is used in function `Spi_WriteIB` to guarantee the atomicity of updating Channel's flags for transmission default data and all data in Channel's buffer if this function is called concurrent in ISR event.

SPI_EXCLUSIVE_AREA_05 is used in function `Spi_SetupEB` to guarantee the atomicity of updating Channel's flags for transmission default data, discard data and Channel's transmit/receive source pointers if this function is called concurrent in ISR event.

SPI_EXCLUSIVE_AREA_06 is used in function `Spi_SyncTransmit` to protect the status of SPI unit belong to the given sequence. Also it protects the global variable which contains the status of the `Spi_SyncTransmit` service. As stated by the Autosar, this service cannot be called when another sequence is during transmission, using this service.

SPI_EXCLUSIVE_AREA_07 is used in function `Spi_SyncTransmit` to to guarantee the atomicity of release SPI units for the given sequence. Also it protects the global variable which contains the status of the `Spi_SyncTransmit` service. As stated by the Autosar, this service cannot be called when another sequence is during transmission, using this service.

SPI_EXCLUSIVE_AREA_08 is used in function `Spi_SyncTransmit` to guarantee the atomicity of updating SPI unit status in `Lpspi_Ip_SyncTransmit` due to `Lpspi_Ip_AsyncTransmit`, `Lpspi_Ip_AsyncTransmitFast` or `Lpspi_Ip_Cancel` called in ISR event.

SPI_EXCLUSIVE_AREA_09 is used in function `Spi_AsyncTransmit` to guarantee the atomicity of updating SPI unit status in `Lpspi_Ip_AsyncTransmit` due to `Lpspi_Ip_AsyncTransmitFast`, `Lpspi_Ip_SyncTransmit` or `Lpspi_Ip_Cancel` called in ISR event.

SPI_EXCLUSIVE_AREA_10 is used in function `Spi_Cancel` to guarantee the atomicity of updating of CR register and updating SPI unit status in `Lpspi_Ip_Cancel` due to `Lpspi_Ip_SyncTransmit`, `Lpspi_Ip_AsyncTransmit`, `Lpspi_Ip_AsyncTransmitFast`, `Lpspi_TransmitTxInit`, `Lpspi_Ip_TransferProcess`, `Lpspi_Ip_IrqTxDmaHandler` and `Lpspi_Ip_IrqRxDmaHandler` may be called in ISR event.

SPI_EXCLUSIVE_AREA_11 is used in function `Spi_SyncTransmit` to guarantee the atomicity of updating of CR register in `Lpspi_TransmitTxInit` due to `Lpspi_Ip_AsyncTransmitFast`, `Lpspi_Ip_Cancel`, `Lpspi_Ip_TransferProcess`, `Lpspi_Ip_IrqTxDmaHandler` and `Lpspi_Ip_IrqRxDmaHandler` may be called in ISR event.

SPI_EXCLUSIVE_AREA_11 is used in function `Spi_AsyncTransmit` to guarantee the atomicity of updating of CR register in `Lpspi_TransmitTxInit` due to `Lpspi_Ip_AsyncTransmitFast`, `Lpspi_Ip_Cancel`, `Lpspi_Ip_TransferProcess`, `Lpspi_Ip_IrqTxDmaHandler` and `Lpspi_Ip_IrqRxDmaHandler` may be called in ISR event.

SPI_EXCLUSIVE_AREA_12 is used in function `Spi_AsyncTransmit` to guarantee the atomicity of updating of CR register in `Lpspi_Ip_TransferProcess` due to `Lpspi_Ip_Cancel`, `Lpspi_TransmitTxInit`, `Lpspi_Ip_AsyncTransmitFast`, `Lpspi_Ip_IrqTxDmaHandler` and `Lpspi_Ip_IrqRxDmaHandler` may be called in ISR event.

SPI_EXCLUSIVE_AREA_13 is used in function `Spi_AsyncTransmit` to guarantee the atomicity of updating of CR register in `Lpspi_Ip_IrqTxDmaHandler` due to `Lpspi_Ip_Cancel`, `Lpspi_TransmitTxInit`, `Lpspi_Ip_AsyncTransmitFast`, `Lpspi_Ip_TransferProcess` and `Lpspi_Ip_IrqRxDmaHandler` may be called in ISR event.

SPI_EXCLUSIVE_AREA_14 is used in function `Spi_AsyncTransmit` to guarantee the atomicity of updating of CR register in `Lpspi_Ip_IrqRxDmaHandler` due to `Lpspi_Ip_Cancel`, `Lpspi_TransmitTxInit`, `Lpspi_Ip_AsyncTransmitFast`, `Lpspi_Ip_TransferProcess` and `Lpspi_Ip_IrqTxDmaHandler` may be called in ISR event.

SPI_EXCLUSIVE_AREA_15 is used in function `Spi_AsyncTransmit` to guarantee the atomicity of updating of CR register and updating SPI unit status in `Lpspi_Ip_AsyncTransmitFast` due to `Lpspi_Ip_SyncTransmit`, `Lpspi_TransmitTxInit`, `Lpspi_Ip_Cancel`, `Lpspi_Ip_TransferProcess`, `Lpspi_Ip_IrqTxDmaHandler` and `Lpspi_Ip_IrqRxDmaHandler` may be called in ISR event.

SPI_EXCLUSIVE_AREA_16 is used in function Spi_Cancel to guarantee the atomicity updating SPI unit status in Flexio_Spi_Ip_Cancel due to Flexio_Spi_Ip_SyncTransmit or Flexio_Spi_Ip_AsyncTransmit may be called in ISR event.

SPI_EXCLUSIVE_AREA_17 is used in function Spi_SyncTransmit to guarantee the atomicity updating SPI unit status in Flexio_Spi_Ip_SyncTransmit due to Flexio_Spi_Ip_AsyncTransmit or Flexio_Spi_Ip_Cancel may be called in ISR event.

SPI_EXCLUSIVE_AREA_18 is used in function Spi_AsyncTransmit to guarantee the atomicity updating SPI unit status in Flexio_Spi_Ip_AsyncTransmit due to Flexio_Spi_Ip_SyncTransmit or Flexio_Spi_Ip_Cancel may be called in ISR event.

Exclusive Areas implemented in Low level driver layer (IPL)

SPI_EXCLUSIVE_AREA_08 is used in function Lpspi_Ip_SyncTransmit to guarantee the atomicity of updating SPI unit status due to Lpspi_Ip_AsyncTransmit, Lpspi_Ip_AsyncTransmitFast or Lpspi_Ip_Cancel called in ISR event.

SPI_EXCLUSIVE_AREA_09 is used in function Lpspi_Ip_AsyncTransmit to guarantee the atomicity of updating SPI unit status due to Lpspi_Ip_AsyncTransmitFast, Lpspi_Ip_SyncTransmit or Lpspi_Ip_Cancel called in ISR event.

SPI_EXCLUSIVE_AREA_10 is used in function Lpspi_Ip_Cancel to guarantee the atomicity of updating of CR register and updating SPI unit status due to Lpspi_Ip_SyncTransmit, Lpspi_Ip_AsyncTransmit, Lpspi_Ip_AsyncTransmitFast, Lpspi_TransmitTxInit, Lpspi_Ip_TransferProcess, Lpspi_Ip_IrqTxDmaHandler and Lpspi_Ip_IrqRxDmaHandler may be called in ISR event.

SPI_EXCLUSIVE_AREA_11 is used in function Lpspi_Ip_SyncTransmit to guarantee the atomicity of updating of CR register in Lpspi_TransmitTxInit due to Lpspi_Ip_AsyncTransmitFast, Lpspi_Ip_Cancel, Lpspi_Ip_TransferProcess, Lpspi_Ip_IrqTxDmaHandler and Lpspi_Ip_IrqRxDmaHandler may be called in ISR event.

SPI_EXCLUSIVE_AREA_11 is used in function Lpspi_Ip_AsyncTransmit to guarantee the atomicity of updating of CR register in Lpspi_TransmitTxInit due to Lpspi_Ip_AsyncTransmitFast, Lpspi_Ip_Cancel, Lpspi_Ip_TransferProcess, Lpspi_Ip_IrqTxDmaHandler and Lpspi_Ip_IrqRxDmaHandler may be called in ISR event.

SPI_EXCLUSIVE_AREA_12 is used in function Lpspi_Ip_AsyncTransmit to guarantee the atomicity of updating of CR register in Lpspi_Ip_TransferProcess due to Lpspi_Ip_Cancel, Lpspi_TransmitTxInit, Lpspi_Ip_AsyncTransmitFast, Lpspi_Ip_IrqTxDmaHandler and Lpspi_Ip_IrqRxDmaHandler may be called in ISR event.

SPI_EXCLUSIVE_AREA_13 is used in function Lpspi_Ip_AsyncTransmit/Lpspi_Ip_AsyncTransmitFast to guarantee the atomicity of updating of CR register in Lpspi_Ip_IrqTxDmaHandler due to Lpspi_Ip_Cancel, Lpspi_TransmitTxInit, Lpspi_Ip_AsyncTransmitFast, Lpspi_Ip_TransferProcess and Lpspi_Ip_IrqRxDmaHandler may be called in ISR event.

SPI_EXCLUSIVE_AREA_14 is used in function Lpspi_Ip_AsyncTransmit/Lpspi_Ip_AsyncTransmitFast to guarantee the atomicity of updating of CR register in Lpspi_Ip_IrqRxDmaHandler due to Lpspi_Ip_Cancel, Lpspi_TransmitTxInit, Lpspi_Ip_AsyncTransmitFast, Lpspi_Ip_TransferProcess and Lpspi_Ip_IrqTxDmaHandler may be called in ISR event.

SPI_EXCLUSIVE_AREA_15 is used in function Lpspi_Ip_AsyncTransmitFast to guarantee the atomicity of updating of CR register and updating SPI unit status due to Lpspi_Ip_SyncTransmit, Lpspi_TransmitTxInit, Lpspi_Ip_Cancel, Lpspi_Ip_TransferProcess, Lpspi_Ip_IrqTxDmaHandler and Lpspi_Ip_IrqRxDmaHandler may be called in ISR event.

SPI_EXCLUSIVE_AREA_16 is used in function Flexio_Spi_Ip_Cancel to guarantee the atomicity updating SPI unit status due to Flexio_Spi_Ip_SyncTransmit or Flexio_Spi_Ip_AsyncTransmit may be called in ISR event.

SPI_EXCLUSIVE_AREA_17 is used in function Flexio_Spi_Ip_SyncTransmit to guarantee the atomicity updating SPI unit status due to Flexio_Spi_Ip_AsyncTransmit or Flexio_Spi_Ip_Cancel may be called in ISR event.

Module requirements

SPI_EXCLUSIVE_AREA_18 is used in function Flexio_Spi_Ip_AsyncTransmit to guarantee the atomicity updating SPI unit status in Flexio_Spi_Ip_AsyncTransmit due to Flexio_Spi_Ip_SyncTransmit or Flexio_Spi_Ip_Cancel may be called in ISR event.

Critical Region Exclusive Matrix Below is the table depicting the exclusivity between different critical region IDs from the SPI driver. If there is an “X” in a table, it means that those 2 critical regions cannot interrupt each other.

The critical regions from interrupts are grouped in “ISRs Critical Regions (composed diagram)”. If an exclusive area is “exclusive” with the composed “ISRs Critical Regions (composed diagram)” group, it means that it is exclusive with each one of the ISR critical regions.

Table 5.1 Exclusive Areas

#	A← R← E← A← _← 00	A← R← E← A← _← 01	A← R← E← A← _← 02	A← R← E← A← _← 03	A← R← E← A← _← 04	A← R← E← A← _← 05	A← R← E← A← _← 06	A← R← E← A← _← 07	A← R← E← A← _← 08	A← R← E← A← _← 09	A← R← E← A← _← 10	A← R← E← A← _← 11	A← R← E← A← _← 12	A← R← E← A← _← 13	A← R← E← A← _← 14	A← R← E← A← _← 15	A← R← E← A← _← 16	A← R← E← A← _← 17	A← R← E← A← _← 18	I← S← R← s C← r← i← t← i← c← a← l R← e← g← i← o← n← s (← c← o← m← p← o← s← e← d d← i← a← g← r← a← m)
A← R← E← A← _← 00																				X
A← R← E← A← _← 01																				X
A← R← E← A← _← 02																				X
A← R← E← A← _← 03																				X

#	A↵ R↵ E↵ A↵ ↵↵ 00	A↵ R↵ E↵ A↵ ↵↵ 01	A↵ R↵ E↵ A↵ ↵↵ 02	A↵ R↵ E↵ A↵ ↵↵ 03	A↵ R↵ E↵ A↵ ↵↵ 04	A↵ R↵ E↵ A↵ ↵↵ 05	A↵ R↵ E↵ A↵ ↵↵ 06	A↵ R↵ E↵ A↵ ↵↵ 07	A↵ R↵ E↵ A↵ ↵↵ 08	A↵ R↵ E↵ A↵ ↵↵ 09	A↵ R↵ E↵ A↵ ↵↵ 10	A↵ R↵ E↵ A↵ ↵↵ 11	A↵ R↵ E↵ A↵ ↵↵ 12	A↵ R↵ E↵ A↵ ↵↵ 13	A↵ R↵ E↵ A↵ ↵↵ 14	A↵ R↵ E↵ A↵ ↵↵ 15	A↵ R↵ E↵ A↵ ↵↵ 16	A↵ R↵ E↵ A↵ ↵↵ 17	A↵ R↵ E↵ A↵ ↵↵ 18	I↵ S↵ Rs Crit- i- cal Re- gions (com- posed di- a- gram)
A↵ R↵ E↵ A↵ ↵↵ 04																				X
A↵ R↵ E↵ A↵ ↵↵ 05																				X
A↵ R↵ E↵ A↵ ↵↵ 06																				X
A↵ R↵ E↵ A↵ ↵↵ 07																				X
A↵ R↵ E↵ A↵ ↵↵ 08									X	X	X					X				X
A↵ R↵ E↵ A↵ ↵↵ 09									X	X	X					X				X

Module requirements

#	A↵ R↵ E↵ A↵ ↵ 00	A↵ R↵ E↵ A↵ ↵ 01	A↵ R↵ E↵ A↵ ↵ 02	A↵ R↵ E↵ A↵ ↵ 03	A↵ R↵ E↵ A↵ ↵ 04	A↵ R↵ E↵ A↵ ↵ 05	A↵ R↵ E↵ A↵ ↵ 06	A↵ R↵ E↵ A↵ ↵ 07	A↵ R↵ E↵ A↵ ↵ 08	A↵ R↵ E↵ A↵ ↵ 09	A↵ R↵ E↵ A↵ ↵ 10	A↵ R↵ E↵ A↵ ↵ 11	A↵ R↵ E↵ A↵ ↵ 12	A↵ R↵ E↵ A↵ ↵ 13	A↵ R↵ E↵ A↵ ↵ 14	A↵ R↵ E↵ A↵ ↵ 15	A↵ R↵ E↵ A↵ ↵ 16	A↵ R↵ E↵ A↵ ↵ 17	A↵ R↵ E↵ A↵ ↵ 18	I↵ S↵ Rs Crit- i- cal Re- gions (com- posed di- a- gram)
A↵ R↵ E↵ A↵ ↵ 10									X	X	X	X	X	X	X	X				X
A↵ R↵ E↵ A↵ ↵ 11											X	X	X	X	X	X				X
A↵ R↵ E↵ A↵ ↵ 12											X	X	X	X	X	X				X
A↵ R↵ E↵ A↵ ↵ 13											X	X	X	X	X	X				X
A↵ R↵ E↵ A↵ ↵ 14											X	X	X	X	X	X				X
A↵ R↵ E↵ A↵ ↵ 15									X	X	X	X	X	X	X	X				X

#	A↔ R↔ E↔ A↔ ↔ 00	A↔ R↔ E↔ A↔ ↔ 01	A↔ R↔ E↔ A↔ ↔ 02	A↔ R↔ E↔ A↔ ↔ 03	A↔ R↔ E↔ A↔ ↔ 04	A↔ R↔ E↔ A↔ ↔ 05	A↔ R↔ E↔ A↔ ↔ 06	A↔ R↔ E↔ A↔ ↔ 07	A↔ R↔ E↔ A↔ ↔ 08	A↔ R↔ E↔ A↔ ↔ 09	A↔ R↔ E↔ A↔ ↔ 10	A↔ R↔ E↔ A↔ ↔ 11	A↔ R↔ E↔ A↔ ↔ 12	A↔ R↔ E↔ A↔ ↔ 13	A↔ R↔ E↔ A↔ ↔ 14	A↔ R↔ E↔ A↔ ↔ 15	A↔ R↔ E↔ A↔ ↔ 16	A↔ R↔ E↔ A↔ ↔ 17	A↔ R↔ E↔ A↔ ↔ 18	I↔ S↔ Rs Crit- i- cal Re- gions (com- posed di- a- gram)
A↔ R↔ E↔ A↔ ↔ 16																	X	X	X	X
A↔ R↔ E↔ A↔ ↔ 17																	X	X	X	X
A↔ R↔ E↔ A↔ ↔ 18																	X	X	X	X
I↔ S↔ Rs Crit- i- cal Re- gions (com- posed di- a- gram)	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X

5.2 Exclusive areas not available on this platform

List of exclusive areas which are not available on this platform (or blank if they're all available).

5.3 Peripheral Hardware Requirements

None

5.4 ISR to configure within AutosarOS - dependencies

The following ISRs are used by the SPI driver and need to be assigned to a priority level. The interrupt vector numbers corresponding to INTERRUPT mode is as shown in Table below. For LPSPI, the interrupt occurs each time the TDF or RDF bits in SR register arises. For FLEXIO, the interrupt occurs each time the SSF bits in SHIFTSTAT register arises and the interrupt vector is different between platforms S32K14X and S32K11X. (Note: Unused interrupts shouldn't be configured in the OS.)

Table 5.2 SPI ISRs for S32K14X

ISR Name	Cortex-M4 Vector	NVIC Interrupt ID
Lpspi_Ip_LPSPI_0_IRQHandler	42	26
Lpspi_Ip_LPSPI_1_IRQHandler	43	27
Lpspi_Ip_LPSPI_2_IRQHandler	44	28
MCL_FLEXIO_ISR	85	69

Table 5.3 SPI ISRs for S32K11X

ISR Name	Cortex-M4 Vector	NVIC Interrupt ID
Lpspi_Ip_LPSPI_0_IRQHandler	42	26
Lpspi_Ip_LPSPI_1_IRQHandler	43	27
Lpspi_Ip_LPSPI_2_IRQHandler	44	28
MCL_FLEXIO_ISR	41	25

Note

In case of AUTOSAR_OS_NOT_USED, the compiler option "-DUSE_SW_VECTOR_MODE" must be added to the list of compiler options to be used with interrupt controller configured to be in software vector mode.

5.5 ISR Macro

RTD drivers use the ISR macro to define the functions that will process hardware interrupts. Depending on whether the OS is used or not, this macro can have different definitions.

5.5.1 Without an Operating System

The macro *USING_OS_AUTOSAROS* must not be defined.

5.5.1.1 Using Software Vector Mode

The macro *USE_SW_VECTOR_MODE* must be defined and the ISR macro is defined as: ***#define ISR(IsrName) void IsrName(void)***

In this case, the drivers' interrupt handlers are normal C functions and their prologue/epilogue will handle the context save and restore.

5.5.1.2 Using Hardware Vector Mode

The macro `USE_SW_VECTOR_MODE` must not be defined and the ISR macro is defined as: `#define ISR(IsrName) INTERRUPT_FUNC void IsrName(void)`

In this case, the drivers' interrupt handlers must also handle the context save and restore.

5.5.2 With an Operating System

Please refer to your OS documentation for description of the ISR macro.

5.6 Other AUTOSAR modules - dependencies

- **BASE**: The BASE module contains the common files/definitions needed by all MCAL modules.
- **DEM**: The DEM module is used for enabling reporting of production relevant error status. The API function used is `Dem_SetEventStatus()`.
- **RESOURCE**: Resource module is used to select microcontroller's derivatives.
- **RTE**: The RTE module is needed for implementing data consistency of exclusive areas that are used by SPI module.
- **DET**: The DET module is used for enabling Default Error Tracer detection. The API function used is `Det_ReportError()`. The activation / deactivation of Default Error Tracer detection is configurable using the "SpiDevErrorDetect" configuration parameter.
- **ECUC**: The ECUC module is used for ECU configuration. MCAL modules need ECUC to retrieve the variant information.
- **PORT**: The PORT module is used to configure the port pins with the needed modes, before they are used by the SPI module. For each SPI, the SCK, SOUT, SIN and CSx_y signals need to be configured. In the S32G2XX Reference manual there is an example of the pin configuration. Please refer to the Reference List.
- **MCU**: The MCU driver provides services for basic microcontroller initialization, power down functionality, reset and microcontroller specific functions required by other MCAL software modules. The clocks need to be initialized prior to using the SPI driver. The SPI reference clock is provided by MCU plugin. The clock frequency may affect the Baudrate, Timing between clock and chip select, Timing between chip select and clock, Timing between chip select assertions. The reference is specified by the parameter `SpiGeneral\SpiPhyUnitClockRef`:



Figure 5.1 Spi reference clock provided by MCU plugin

- **MCL**: For each LPSPI in use, a transmit and a receive DMA channel need to be defined and routed through the DMA Multiplexer using MCL plugin. MCL should be initialized before SPI switch to DMA mode. For each FLEXIO_SPI is used, MCL is needed to configure Flexio channels (MOSI, MISO, CS, CLK) and enable/disable Flexio module and ISR handler
- **OS**: Needed to define a mapping between EcuC partitions and EcuC core ids when MultiPartition support is enabled.

5.7 Data Cache Restrictions

In the DMA transfer mode, DMA transfers may issue cache coherency problems. To avoid possible coherency issues when **D-CACHE** is enabled, the user shall ensure that the buffers used as TCD source and destination are allocated

in the **NON-CACHEABLE** area (by means of Spi_Memmap). Otherwise, the SPI driver has some dependencies. User must to put all variables, which were used for transmitter and receiver, to the **NON CACHEABLE** memory section in the RAM zone by the definition **SPI_START_SEC_VAR__INIT_POLICY>__<ALIGNMENT>__NO_CACHEABLE** and **SPI_STOP_SEC_VAR__INIT_POLICY>__<ALIGNMENT>__NO_CACHEABLE**

5.8 User Mode support

- [User Mode configuration in the module](#)
- [User Mode configuration in AutosarOS](#)

Spi module does not include registers protection. So, It is accessible to all registers in any public mode.

5.8.1 User Mode configuration in the module

The Spi can be run in user mode if the following steps are performed:

- Enable **SpiEnableUserModeSupport** from the configuration
- Call the following functions as trusted functions:

Function syntax	Description	Available via
void Lpspi_Ip_SetUserAccess(uint8 Instance)	This function will enable writing all SPI registers under protection in User mode by configuring REG_PROT	Lpspi_Ip_TrustedFunctions.h
void Flexio_Spi_Ip_SetUserAccess(uint8 Instance)	This function will enable writing all FLEXIO_SPI registers under protection in User mode by configuring REG_PROT	Flexio_Spi_Ip_TrustedFunctions.h

5.8.2 User Mode configuration in AutosarOS

When User mode is enabled, the driver may have the functions that need to be called as trusted functions in AutosarOS context. Those functions are already defined in driver and declared in the header `<IpName>_Ip_TrustedFunctions.h`. This header also included all headers files that contains all types definition used by parameters or return types of those functions. Refer the chapter [User Mode configuration in the module](#) for more detail about those functions and the name of header files they are declared inside. Those functions will be called indirectly with the naming convention below in order to AutosarOS can call them as trusted functions.

`Call_<Function_Name>_TRUSTED(parameter1,parameter2,...)`

That is the result of macro expansion `OsIf_Trusted_Call` in driver code:

```
#define OsIf_Trusted_Call[1-6params](name,param1,...,param6) Call_##name##_TRUSTED(param1,...,param6)
```

So, the following steps need to be done in AutosarOS:

- Ensure `MCAL_ENABLE_USER_MODE_SUPPORT` macro is defined in the build system or somewhere global.
- Define and declare all functions that need to call as trusted functions follow the naming convention above in Integration/User code. They need to visible in `Os.h` for the driver to call them. They will do the marshallng of the parameters and call `CallTrustedFunction()` in OS specific manner.

- `CallTrustedFunction()` will switch to privileged mode and call `TRUSTED_<Function_Name>()`.
- `TRUSTED_<Function_Name>()` function is also defined and declared in Integration/User code. It will un-marshalling of the parameters to call `<Function_Name>()` of driver. The `<Function_Name>()` functions are already defined in driver and declared in `<IpName>_Ip_TrustedFunctions.h`. This header should be included in OS for OS call and indexing these functions.

5.9 MultiPartition support

This S32K1XX platform has only one partition, so MultiPartition is not available.

Chapter 6

Main API Requirements

- [Main function calls within BSW scheduler](#)
- [API Requirements](#)
- [Calls to Notification Functions, Callbacks, Callouts](#)

6.1 Main function calls within BSW scheduler

The function `Spi_MainFunction_Handling()` should be called periodically only if polling mode is enabled for `Spi←_AsyncTransmit()` .

6.2 API Requirements

None.

6.3 Calls to Notification Functions, Callbacks, Callouts

Call-back Notifications: None

User Notification: The SPI Handler & Driver provides notifications per job and sequence in asynchronous mode. The notifications can be configured as pointers to user defined functions. If notification is not desired, the appropriate `EndNotification` field shall be left blank. For asynchronous transmissions, job and sequences notifications are performed before the scheduling of the next job (contrary to the recommendation given by SWS_Spi_00088) . In this way, calls like `Spi_SetupIB()` or `Spi_WriteIB()` can be targeted on the next schedulable jobs, before the starting of the job transfer.

Chapter 7

Memory allocation

- [Sections to be defined in Spi_MemMap.h](#)
- [Linker command file](#)

7.1 Sections to be defined in Spi_MemMap.h

Section name	Type of section	Description
SPI_START_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	Start of Memory Section for Config Data when the SIZE (alignment) does not fit the criteria of 8,16 or 32 bit.
SPI_STOP_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	End of Memory Section for Config Data
SPI_START_SEC_CONST_32	Configuration Data	Start of Memory Section for constant data which have to be aligned to 32 bit.
SPI_STOP_SEC_CONST_32	Configuration Data	End of Memory Section for constant data.
SPI_START_SEC_CONST_UNSPECIFIED	Configuration Data	Start of Memory Section for constant data when the SIZE (alignment) does not fit the criteria of 8,16 or 32 bit.
SPI_STOP_SEC_CONST_UNSPECIFIED	Configuration Data	End of Memory Section for constant data.
SPI_START_SEC_CODE	Code	Start of memory Section for Code
SPI_STOP_SEC_CODE	Code	End of memory Section for Code
SPI_START_SEC_VAR_Cleared_32	Variables	Used for variables which have to be aligned to 32 bit. For instance used for variables of size 32 bit or used for composite data types: arrays, structures containing elements of maximum 32 bits. These variables are cleared to zero by start-up code.
SPI_STOP_SEC_VAR_Cleared_32	Variables End of above section.	End of above section.

Section name	Type of section	Description
SPI_START_SEC_VAR_CLEARED_32_NO_CACHEABLE	Variables	Used for variables which have to be aligned to 32 bit. For instance used for variables of size 32 bit or used for composite data types: arrays, structures containing elements of maximum 32 bits and that have to be stored in a non-cacheable memory section. These variables are cleared to zero by start-up code.
SPI_STOP_SEC_VAR_CLEARED_32_NO_CACHEABLE	Variables End of above section.	End of above section.
SPI_START_SEC_VAR_CLEARED_UNSPECIFIED	Variables	Used for variables, structures, arrays when the SIZE (alignment) does not fit the criteria of 8,16 or 32 bit. These variables are cleared to zero by start-up code.
SPI_STOP_SEC_VAR_CLEARED_UNSPECIFIED	Variables	End of above section.
SPI_START_SEC_VAR_CLEARED_UNSPECIFIED_NO_CACHEABLE	Variables	Used for variables, structures, arrays when the SIZE (alignment) does not fit the criteria of 8,16 or 32 bit, and that have to be stored in a non-cacheable memory section. These variables are cleared to zero by start-up code.
SPI_STOP_SEC_VAR_CLEARED_UNSPECIFIED_NO_CACHEABLE	Variables	End of above section.
SPI_START_SEC_VAR_INIT_UNSPECIFIED	Variables	Used for variables, structures, arrays when the SIZE (alignment) does not fit the criteria of 8,16 or 32 bit.
SPI_STOP_SEC_VAR_INIT_UNSPECIFIED	Variables	End of above section.

7.2 Linker command file

Memory shall be allocated for every section defined in the driver's "<Module>_MemMap.h.

Chapter 8

Integration Steps

This section gives a brief overview of the steps needed for integrating this module:

1. Generate the required module configuration(s). For more details refer to section [Files Required for Compilation](#)
2. Allocate the proper memory sections in the driver's memory map header file ("`<Module>_MemMap.h`") and linker command file. For more details refer to section [Sections to be defined in `<Module>\_MemMap.h`](#)
3. Compile & build the module with all the dependent modules. For more details refer to section [Building the Driver](#)

Chapter 9

External assumptions for driver

The section presents requirements that must be complied with when integrating the SPI driver into the application.

External Assumption Req ID	External Assumption Text
SWS_Spi_00244	The SPI Handler/Driver module does not take care of setting the registers which configure the clock, prescaler(s) and PLL in its init function. This has to be done by the MCU module [REF].
SWS_Spi_00052	For the IB Channels, the Handler/Driver shall provide the buffering but it is not able to take care of the consistency of the data in the buffer during transmission. The size of the Channel buffer is fixed.
SWS_Spi_00257	The SPI Handler/Driver is not able to prevent the overwriting of these "transmit" buffers by users during transmissions.
SWS_Spi_00053	For EB Channels the application shall provide the buffering and shall take care of the consistency of the data in the buffer during transmission.
SWS_Spi_00280	The buffer provided by the application for the SPI Handler Driver may have a different size. Note: This refers in the context of External Buffer
SWS_Spi_00084	If different Jobs (and consequently also Sequences) have common Channels, the SPI Handler/Driver's environment shall ensure that read and/or write functions are not called during transmission.
SWS_Spi_00121	The SPI Handler/Driver's environment shall configure the Spi_InterruptibleSeqAllowed parameter (ON / OFF) in order to select which kind of Sequences the SPI Handler/Driver manages.
SWS_Spi_00080	When using Interruptible Sequences, the caller must be aware that if the multiple Sequences access the same Channels, the data for these Channels may be overwritten by the highest priority Job accessing each Channel.
SWS_Spi_00298	The operation Spi_Init is Non Re-entrant.
SWS_Spi_00300	The operation Std_ReturnType Spi_DeInit() is Non Re-entrant.
SWS_Spi_00173	The SPI Handler/Driver's environment shall call the function Spi_AsyncTransmit after a function call of Spi_SetupEB for EB Channels or a function call of Spi_WriteIB for IB Channels but before the function call Spi_ReadIB.
SWS_Spi_00027	The SPI Handler/Driver's environment shall call the function Spi_ReadIB after a Transmit method call to have relevant data within IB Channel.
SWS_Spi_00037	The SPI Handler/Driver's environment shall call the Spi_SetupEB function once for each Channel with EB declared before the SPI Handler/Driver's environment calls a Transmit method on them.

External Assumption Req ID	External Assumption Text
SWS_Spi_00038	The SPI Handler/Driver's environment shall call the function Spi_GetJob↔Result to inquire whether the Job transmission has succeeded (SPI_JOB↔_OK) or failed (SPI_JOB_FAILED).
SWS_Spi_00042	The SPI Handler/Driver's environment shall call the function Spi_Get↔SequenceResult to inquire whether the full Sequence transmission has succeeded (SPI_SEQ_OK) or failed (SPI_SEQ_FAILED).
SWS_Spi_00287	The SPI Handler/Driver's environment shall call this function to inquire whether the specified SPI Hardware microcontroller peripheral is SPI_IDLE or SPI_BUSY. Note: This requirement refers to Spi_GetHWUnitStatus()
SWS_Spi_00335	The operation Spi_SetAsyncMode is Non Re-entrant.
SWS_Spi_00265	For implement the call back function other modules are required to provide the routines in the expected manner.
SWS_Spi_00048	The callback notifications Spi_JobEndNotification and Spi_SeqEnd↔Notification shall have no parameters and no return value.
SWS_Spi_00085	It is allowed to use the following API calls within the SPI callback notifications: - Spi_ReadIB - Spi_WriteIB - Spi_SetupEB - Spi_GetJob↔Result - Spi_GetSequenceResult - Spi_GetHWUnitStatus - Spi_CancelAll other SPI Handler/Driver API calls are not allowed.
SWS_Spi_00340	The operation SpiJobEndNotification is Re-entrant.
SWS_Spi_00341	The operation SpiSeqEndNotification is Re-entrant.
SWS_Spi_00077	To transmit a variable number of data, it is mandatory to call the Spi_↔SetupEB function to store new parameters within SPI Handler/Driver before each Spi_AsyncTransmit function call.
SWS_Spi_00078	To transmit a constant number of data, it is only mandatory to call the Spi_↔SetupEB function to store parameters within SPI Handler/Driver before the first Spi_AsyncTransmit function call.
SWS_Spi_00235	If not applicable, the SPI Handler/Driver module's environment shall pass a NULL pointer to the function Spi_Init.
EA_RTD_00071	If interrupts are locked, a centralized function pair to lock and unlock interrupts shall be used.
EA_RTD_00081	The integrator shall assure that <MSN>_Init() and <MSN>_DeInit() functions do not interrupt each other.
EA_RTD_00082	When caches are enabled and data buffers are allocated in cacheable memory regions the buffers involved in DMA transfer shall be aligned with both start and end to cache line size. Note: Rationale: This ensures that no other buffers/variables compete for the same cache lines.
EA_RTD_00093	The application shall define EcucCoreIDs in a compact/consecutive order, starting from zero.
EA_RTD_00096	The application shall pass the correct initialization pointer, specific to the partition in which the driver is to be used.
EA_RTD_00106	Standalone IP configuration and HL configuration of the same driver shall be done in the same project
EA_RTD_00107	The integrator shall use the IP interface only for hardware resources that were configured for standalone IP usage. Note: The integrator shall not directly use the IP interface for hardware resources that were allocated to be used in HL context.
EA_RTD_00108	The integrator shall use the IP interface to a build a CDD, therefore the BSWMD will not contain reference to the IP interface

External assumptions for driver

External Assumption Req ID	External Assumption Text
EA_RTD_00113	When RTD drivers are integrated with AutosarOS and User mode support is enabled, the integrator shall assure that the definition and declaration of all RTD functions needed to be called as trusted functions follow the naming convention Call<Function_Name>TRUSTED(parameter1,parameter2,...) in Integration/User code. They need to be visible in Os.h for the driver to call them. They will call RTD <Function_Name>() as trusted functions in OS specific manner.

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Limited warranty and liability - Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.

In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.

Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes - NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use in automotive applications - This NXP product has been qualified for use in automotive applications. If this product is used by customer in the development of, or for incorporation into, products or services (a) used in safety critical applications or (b) in which failure could lead to death, personal injury, or severe physical or environmental damage (such products and services hereinafter referred to as "Critical Applications"), then customer makes the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, safety, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP. As such, customer assumes all risk related to use of any products in Critical Applications and NXP and its suppliers shall not be liable for any such use by customer. Accordingly, customer will indemnify and hold NXP harmless from any claims, liabilities, damages and associated costs and expenses (including attorneys' fees) that NXP may incur related to customer's incorporation of any product in a Critical Application.

Applications - Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.

Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Terms and conditions of commercial sale - NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at <http://www.nxp.com/profile/terms>, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

Export control - This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Translations — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately.

Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.

NXP has a Product Security Incident Response Team (PSIRT) (reachable at <mailto:PSIRT@nxp.com>) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

NXP B.V. - NXP B.V. is not an operating company and it does not distribute or sell products.